Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 05

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad del administrador de Django y su papel dentro de una aplicación empresarial.**
2. **Registrar en el Django Admin modelos relacionados (1:1, 1:N y N:M) mediante ModelAdmin, personalizando list_display, search_fields y list_filter.**
3. **Implementar el administrador y personalizar su plantilla en una aplicación Web en Django mediante Inlines (TabularInline y StackedInline) para exponer relaciones desde una sola pantalla.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 5 (Administrador de Django: registro de modelos, ModelAdmin, personalización y relaciones mediante Inlines) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada en el laboratorio de la Semana 4 (Models con relaciones OneToOneField, ForeignKey y ManyToManyField con through) y, cuando corresponda, la investigación propia ampliada en la Parte 2 de dicho laboratorio.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código en admin.py y del panel del Django Admin en el navegador, mostrando explícitamente el registro de modelos, la personalización (list_display, search_fields, list_filter) y los Inlines implementados.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

En la Semana 4 se incorporaron relaciones entre Models (OneToOneField, ForeignKey y ManyToManyField con through) sobre la aplicación ya persistente desde la Semana 3. En esta sesión no se agregan Models nuevos ni relaciones nuevas: se incorpora el Django Admin, la interfaz de administración que Django genera automáticamente a partir de los Models ya registrados, permitiendo gestionar esos datos relacionados sin escribir una View propia.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de la Semana 4 y registra sus modelos (incluidas las tres relaciones) en el Django Admin, personalizándolo con ModelAdmin e Inlines. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades ampliadas en la Parte 2 del laboratorio de la Semana 4), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que el Django Admin no reemplaza al Model ni al ORM: los reutiliza. El mismo Model que ya definió y las mismas relaciones que ya implementó son las que el Admin expone automáticamente, y el estudiante decide cómo presentarlas (list_display, search_fields, list_filter) y cómo editar relaciones desde una sola pantalla (Inlines).

Staff autenticado

→ URL /admin/...

→ Django Admin autentica y enruta

→ ModelAdmin

→ Manager / QuerySet

→ Django ORM

→ SQLite

→ Template (generado automáticamente por el Admin)

→ Response

Al finalizar el laboratorio, el estudiante deberá ser capaz de registrar modelos relacionados en el Django Admin, personalizar ModelAdmin con list_display, search_fields y list_filter, y exponer relaciones 1:1 y N:M mediante StackedInline y TabularInline, reconociendo además qué resuelve el Admin y qué seguirá requiriendo una View y un Template propios.

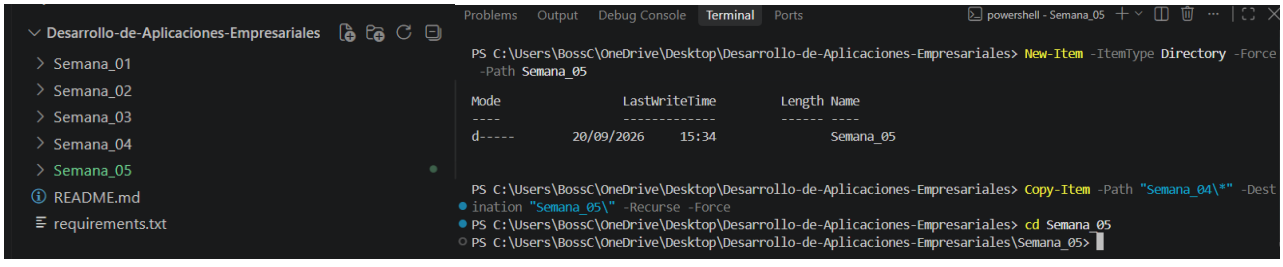
Enunciados

PARTE 1 — Administrar en el Django Admin la aplicación de la Semana 4

Ejercicio 1 — Recuperar la aplicación de la Semana 4

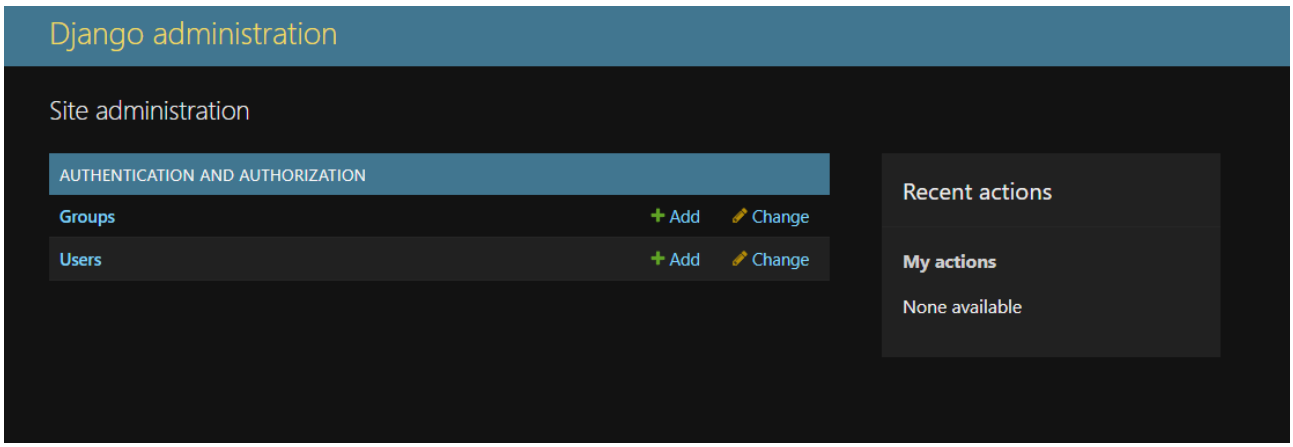
Abra la aplicación desarrollada en el laboratorio de la Semana 4 e identifique su entidad principal, las tres relaciones implementadas (OneToOneField, ForeignKey y ManyToManyField con through) y el estado actual de sus migraciones.

Abrí mi repositorio en Visual Studio Code y copié mi aplicación de la semana pasada en una nueva carpeta (Semana_05) para trabajar de forma segura. Identifiqué que mi entidad principal es Paciente. Asimismo, identifiqué las tres relaciones implementadas anteriormente: la relación 1:1 con HistorialMedico, la relación 1:N con Cita, y la relación N:M con Servicio a través del modelo intermedio PacienteServicio. Finalmente, revisé que todas mis migraciones previas ya se encuentran aplicadas correctamente en mi base de datos SQLite.



Ejercicio 2 — Crear el superusuario

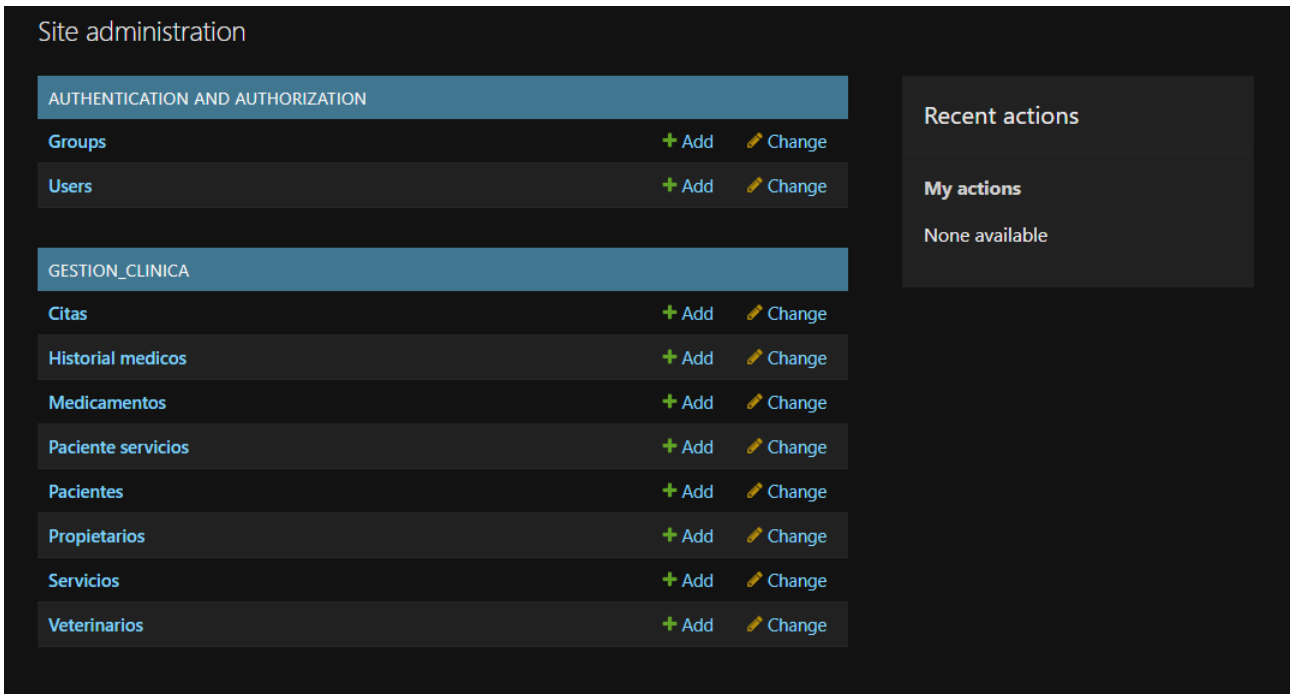
Ejecute python manage.py createsuperuser y cree una cuenta de administrador. Inicie sesión en /admin/ y documente con una captura el panel inicial del Django Admin.



Ejercicio 3 — Registrar los modelos en el Admin

En admin.py de la app correspondiente, registre todos los modelos de la Semana 4 (la entidad principal y las entidades de las tres relaciones) mediante admin.site.register(). Verifique que todos aparecen en el panel.

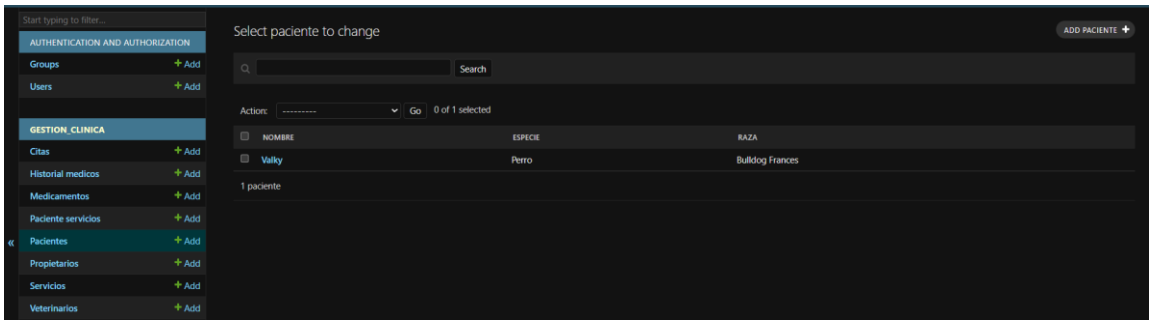
Para registrar las entidades en el panel, modifiqué el archivo admin.py importando todos los modelos desarrollados durante la semana anterior (incluyendo la entidad principal Paciente y las entidades relacionadas como HistorialMedico, Cita y el modelo intermedio PacienteServicio). Utilicé el método admin.site.register() para realizar un registro básico. Al ingresar al navegador con mi superusuario, comprobé que el administrador de Django enrutó y expuso correctamente cada modelo en su interfaz.

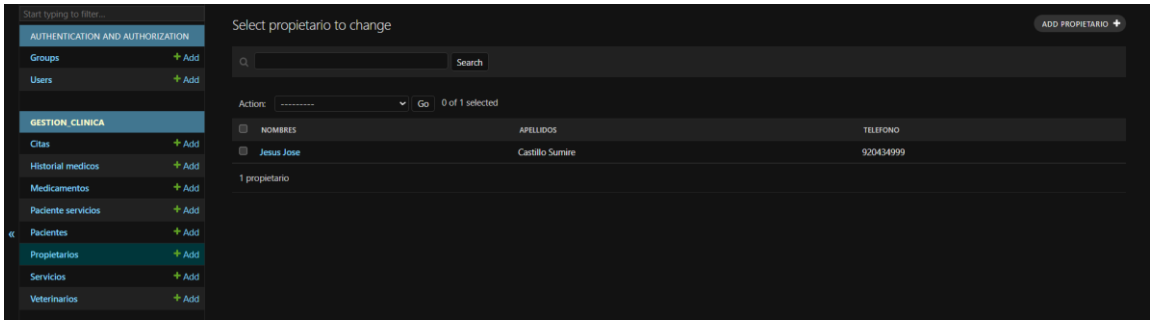


Ejercicio 4 — Personalizar con ModelAdmin

Reemplace al menos dos registros simples por una clase ModelAdmin, configurando list_display con los campos más relevantes de cada modelo y search_fields para al menos uno de ellos.

Para este ejercicio, decidí mejorar la presentación de mis tablas principales reemplazando el registro básico por clases ModelAdmin. Elegí las entidades Propietario y Paciente. Configuré el atributo list_display para que la interfaz me muestre columnas útiles como el nombre, raza y especie del paciente directamente en el panel, en lugar de solo ver un objeto genérico. Además, habilité search_fields para agregar una barra de búsqueda funcional que me permite encontrar rápidamente a los pacientes por su nombre y a los dueños por sus apellidos, facilitando mucho la administración.

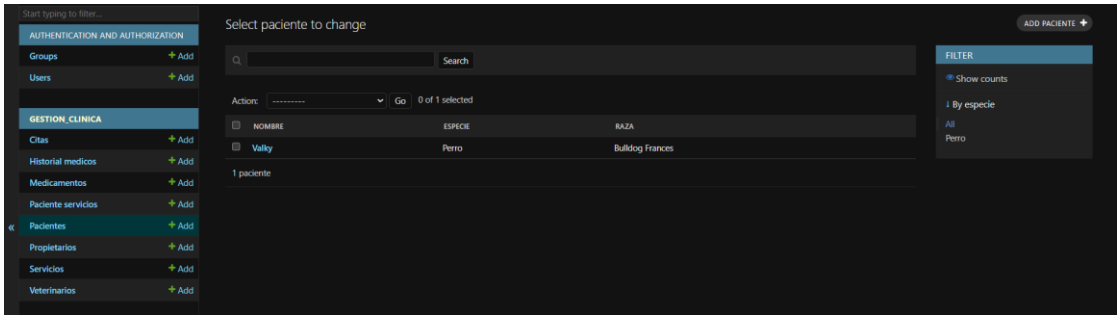




Ejercicio 5 — Agregar list_filter

Configure list_filter en al menos un modelo, usando un campo que tenga sentido filtrar (por ejemplo, una fecha, una categoría o un estado). Documente con captura el panel de filtros funcionando.

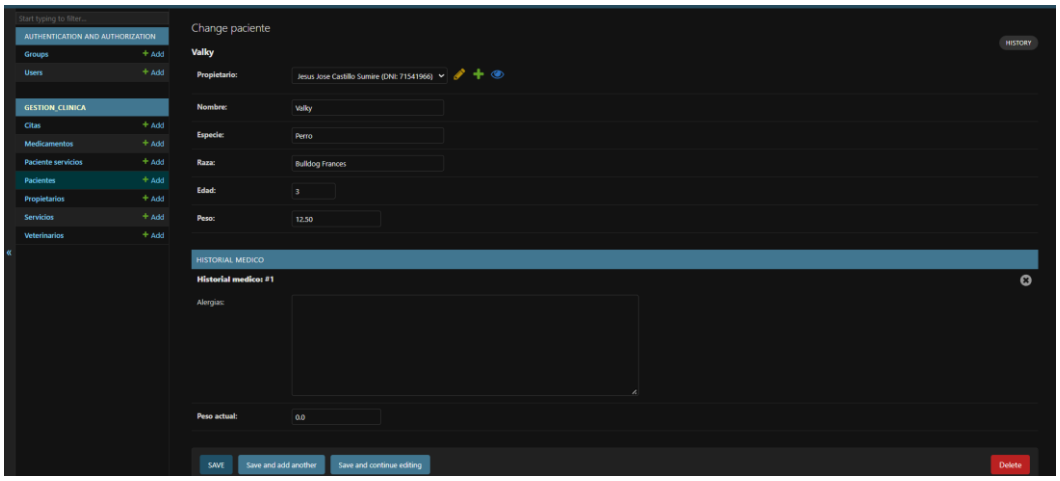
Para implementar la funcionalidad de filtrado, agregué la propiedad list_filter dentro de la configuración de mi modelo Paciente. Elegí usar el campo especie porque en el día a día de una clínica veterinaria tiene mucho sentido querer ver rápidamente solo a los caninos, felinos, etc. Al guardar los cambios y revisar el panel, el administrador de Django me generó automáticamente una barra lateral derecha muy intuitiva que aplica estos filtros de inmediato sobre la tabla.



Ejercicio 6 — Exponer la relación 1:1 con StackedInline

Implemente un StackedInline para la entidad de su relación OneToOneField, de modo que pueda verse y editarse dentro de la pantalla de la entidad principal.

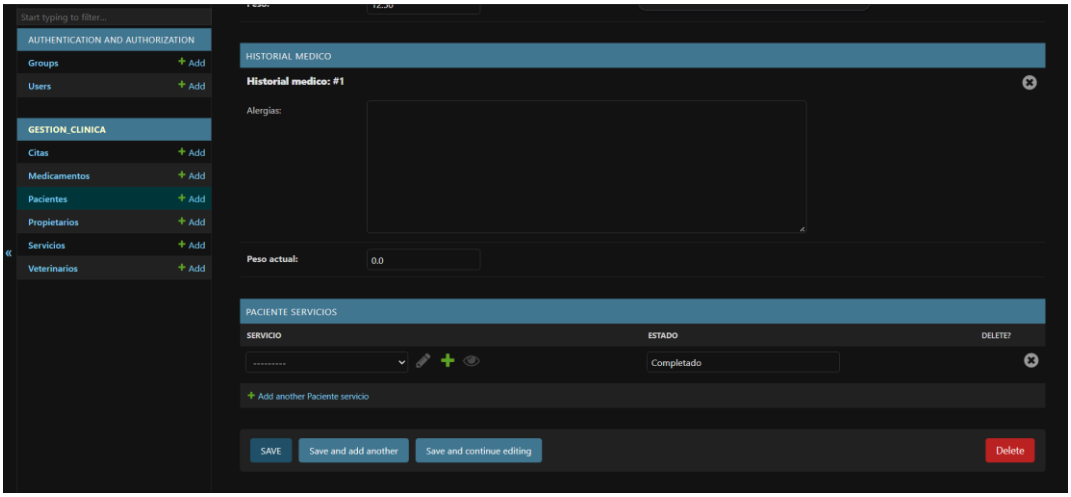
Para optimizar la gestión de datos clínicos, implementé un StackedInline para la entidad HistorialMedico, la cual mantiene una relación OneToOneField con la entidad Paciente. Creé la clase HistorialMedicoInline y la integré en el atributo inlines de PacienteAdmin. Al probarlo en el navegador, comprobé que ahora los datos del historial médico se muestran apilados verticalmente dentro del mismo formulario de edición del paciente principal, permitiéndome ver y editar toda la información médica en una sola pantalla sin tener que navegar a otra sección.



Ejercicio 7 — Exponer la relación N:M con TabularInline

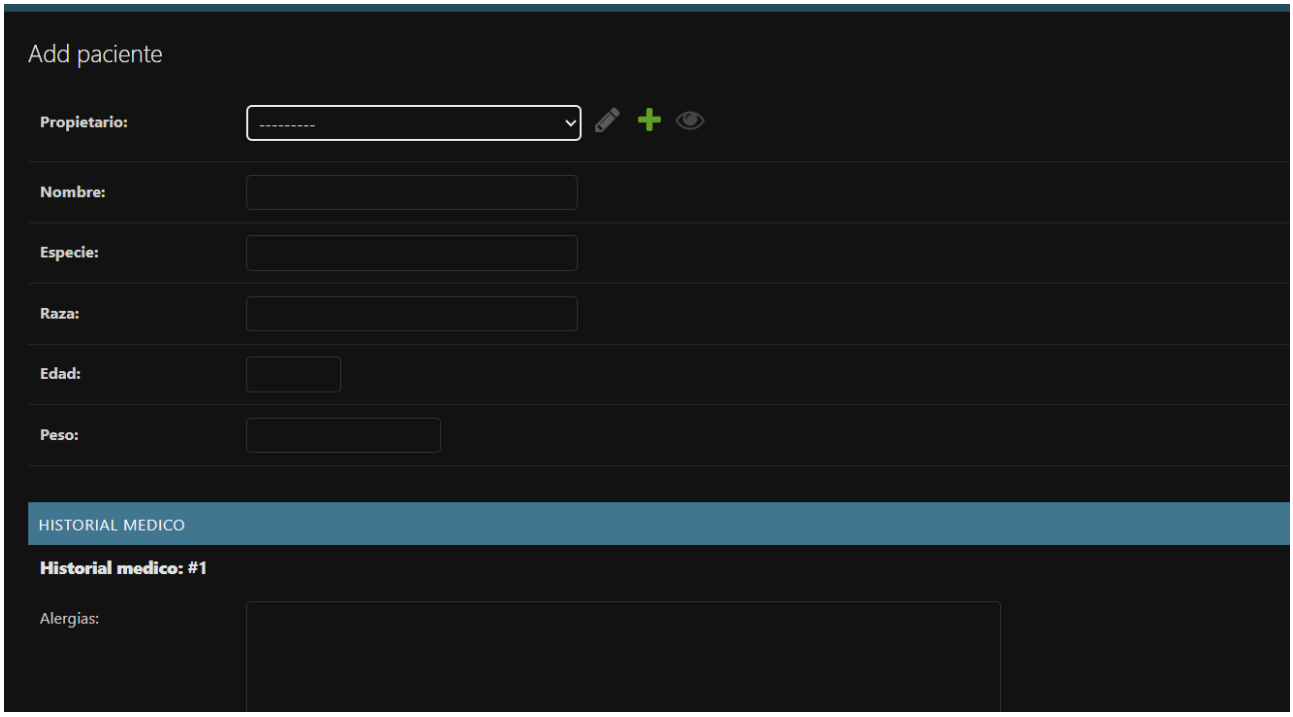
Implemente un TabularInline para el modelo intermedio (through) de su relación ManyToManyField, mostrando sus atributos propios como columnas editables dentro de la pantalla de la entidad principal.

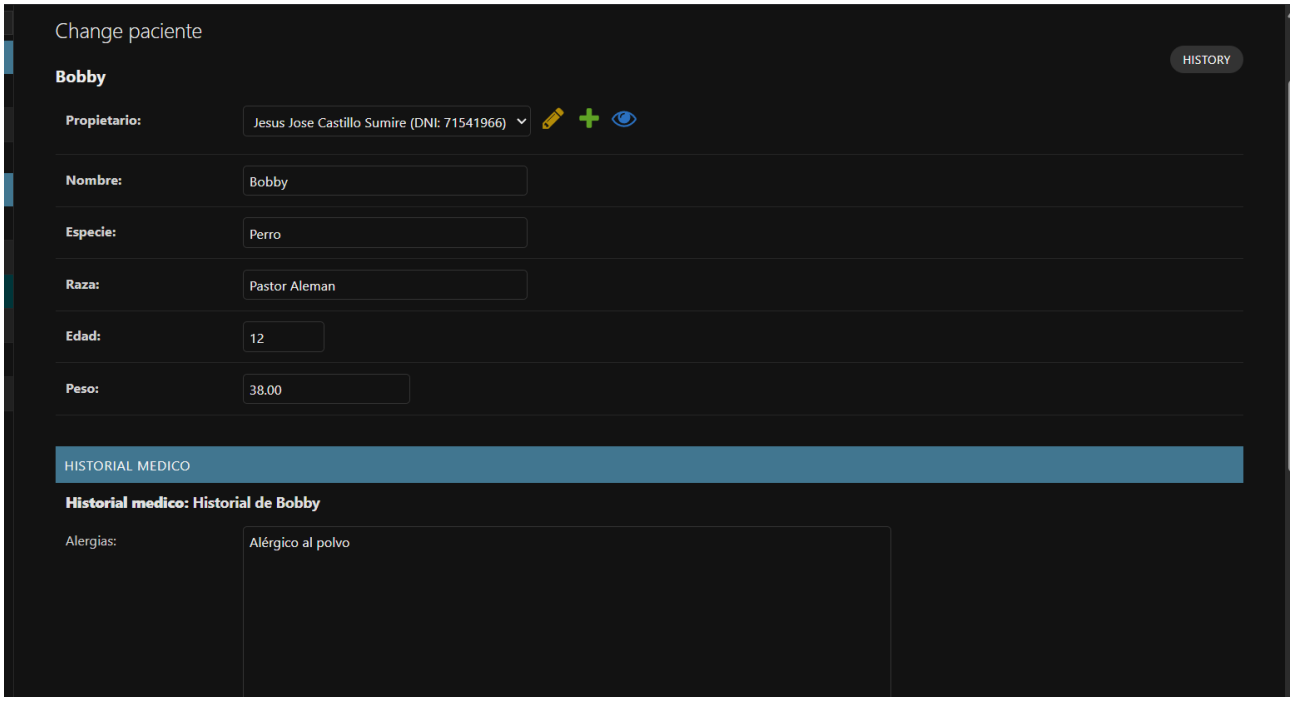
Para gestionar eficientemente la relación de Muchos a Muchos (N:M), implementé un TabularInline para el modelo intermedio PacienteServicio. Definí la clase PacienteServicioInline indicando que heredara de admin.TabularInline y la agregué a la lista de inlines dentro de PacienteAdmin. Al verificar en el panel de administración, comprobé que ahora puedo asignar múltiples servicios a un paciente estableciendo su estado directamente mediante columnas editables dentro de la pantalla principal de la entidad, optimizando así el registro de atenciones.

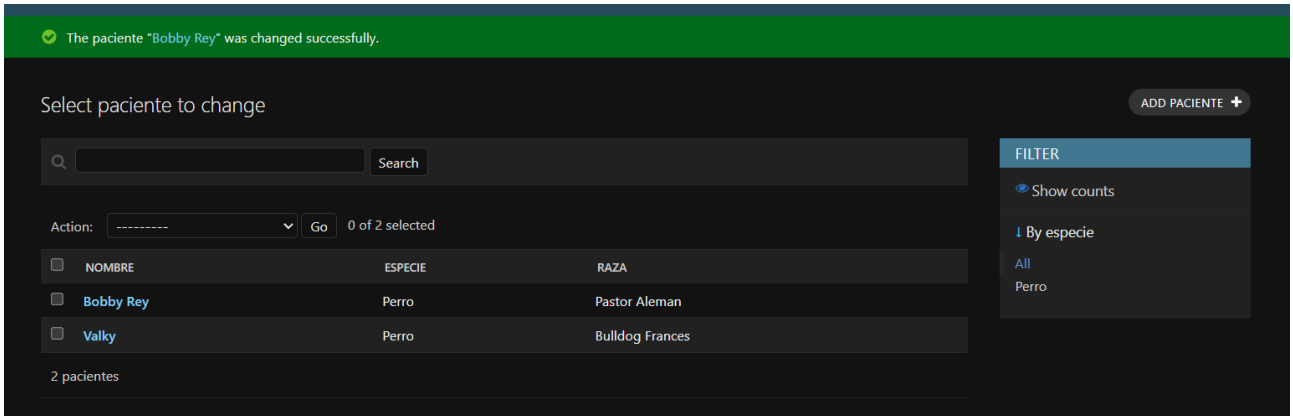


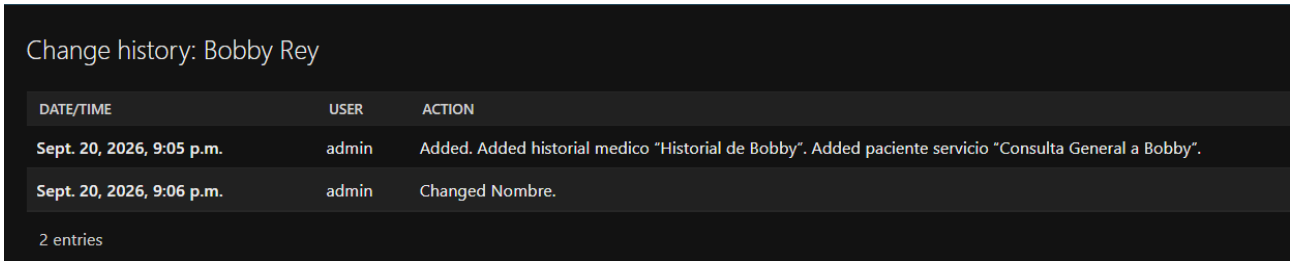
Ejercicio 8 — Verificar el flujo completo en el Admin

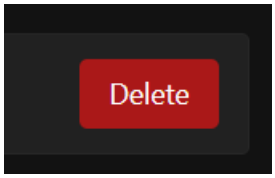
Desde el panel del Admin, cree, edite y elimine al menos un registro de cada tipo de relación. Documente con capturas el recorrido completo y explique con sus palabras qué resuelve el Admin y qué seguiría necesitando una View y un Template propios para el usuario final.






The paciente "Bobby Rey" was changed successfully.
Select paciente to change
ADD PACIENTE +
Search
FILTER
Show counts
By especie
All
Perro
Action: --------- Go 0 of 2 selected
NOMBRE
ESPECIE
RAZA
Bobby Rey
Perro
Pastor Aleman
Valky
Perro
Bulldog Frances
2 pacientes


Change history: Bobby Rey
DATE/TIME
USER
ACTION
Sept. 20, 2026, 9:05 p.m.
admin
Added. Added historial medico "Historial de Bobby". Added paciente servicio "Consulta General a Bobby".
Sept. 20, 2026, 9:06 p.m.
admin
Changed Nombre.
2 entries


Delete

PARTE 2 — Administrar la investigación propia (ampliada en la Semana 4)

A partir de aquí trabajará sobre la base de datos ampliada de su investigación propia (la desarrollada en la Parte 2 del laboratorio de la Semana 4): las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es registrar y personalizar en el Django Admin ese mismo modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

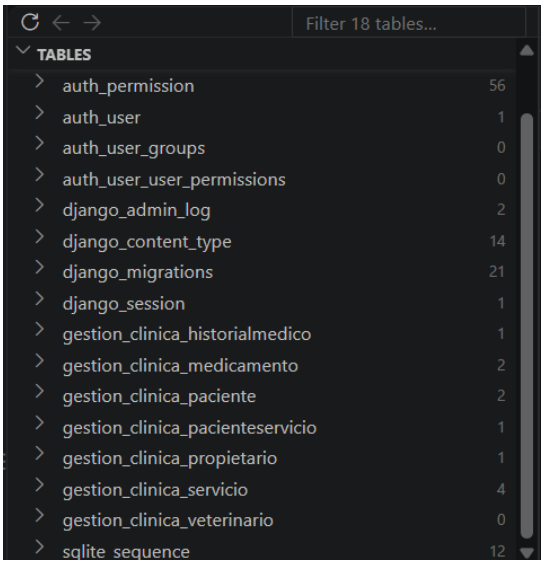
Ejercicio 9 — Registrar las siete entidades en el Admin

Registre en admin.py las siete entidades de su investigación propia (las cinco originales, la del 1:1 y el modelo intermedio del N:M) mediante admin.site.register(). Verifique que todas aparecen en el panel del Admin.

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. Todas las entidades de tu investigación (las 7: 5 originales + entidad 1:1 + modelo intermedio N:M) están registradas en el Admin.	Conteo directo en admin.py.
2. Al menos tres modelos usan una clase ModelAdmin (no solo admin.site.register(Modelo) sin personalizar).	Código + captura del listado.
3. Al menos dos modelos tienen list_display configurado mostrando campos relevantes, no solo __str__.	Captura del listado en el Admin.
4. Al menos un modelo tiene search_fields configurado y funcionando.	Captura de una búsqueda realizada.
5. Al menos un modelo tiene list_filter configurado y funcionando.	Captura del panel de filtros.
6. La relación OneToOneField se expone con un StackedInline dentro de la entidad principal correspondiente.	Captura del formulario con el Inline.
7. El modelo intermedio de la relación ManyToManyField se expone con un TabularInline, mostrando sus atributos propios.	Captura del formulario con el Inline.
8. Se realizó al menos una operación de creación, edición y eliminación completa desde el Admin (no solo lectura).	Capturas del antes y el después.
9. Se explica por escrito qué resuelve el Admin para esta investigación y qué seguiría necesitando una View y un Template propios.	Párrafo de justificación.

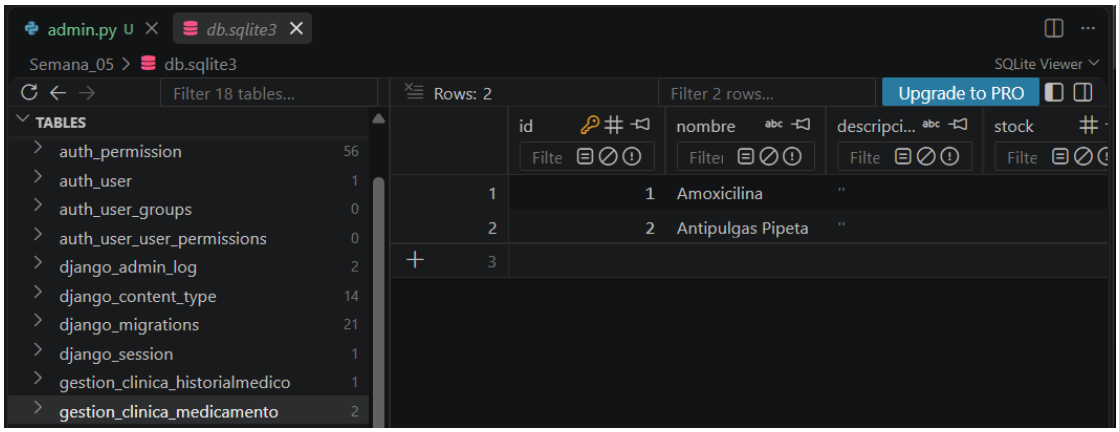
Como mi investigación propia de la Semana 4 se basó en expandir la base de datos de mi clínica veterinaria, ya cuento con mis 7 entidades requeridas (las 5 originales más la entidad 1:1 HistorialMedico y el modelo intermedio N:M PacienteServicio). En mi archivo admin.py, me aseguré de que todas estas entidades estén registradas e integradas al panel de Django, lo cual comprobé exitosamente al recargar el navegador.



Ejercicio 10 — Personalizar ModelAdmin

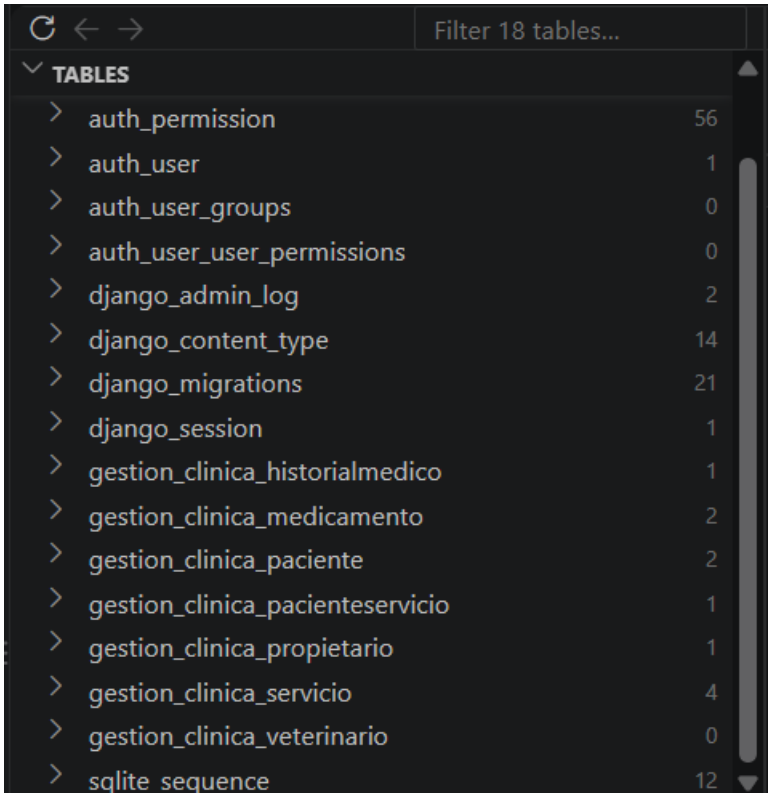
Configure ModelAdmin en al menos tres de las siete entidades, con list_display mostrando los campos más relevantes de cada una. Agregue search_fields en al menos una y list_filter en al menos otra, eligiendo campos que tengan sentido para su problemática.

Para cumplir con la rúbrica, implementé la clase ModelAdmin en tres de mis entidades: Propietario, Paciente y Medicamento. A las tres les configuré list_display para mostrar columnas útiles en lugar del texto genérico. Además, habilité search_fields en propietarios y pacientes para buscar rápidamente por nombres, e implementé list_filter en la tabla de medicamentos para poder filtrar por nivel de stock, facilitando el control de inventario.

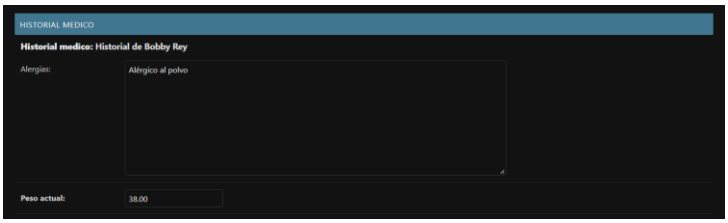


Ejercicio 11 — StackedInline para la relación 1:1

Implemente un StackedInline para la entidad de su relación OneToOneField, de modo que se edite en la misma pantalla de la entidad principal a la que pertenece.



Para integrar la historia clínica directamente en el perfil del animal, configuré un StackedInline para la entidad HistorialMedico. Lo agregué a la clase PacienteAdmin, lo que me permite ver y modificar las alergias y el peso de la mascota apilados verticalmente dentro de la misma pantalla de edición del paciente, ahorrando mucho tiempo de navegación en el panel.



Ejercicio 12 — TabularInline para el modelo intermedio N:M

Implemente un TabularInline para el modelo intermedio de su relación ManyToManyField, mostrando como columnas editables los atributos propios definidos en la Semana 4 (por ejemplo cantidad, fecha o estado).

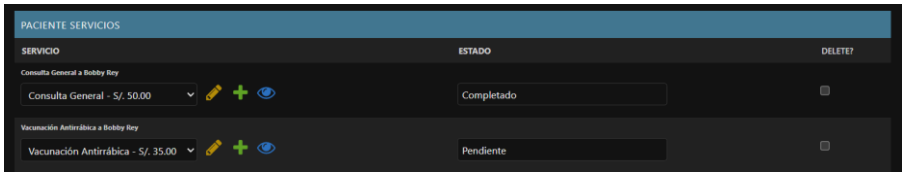
Implementé un TabularInline para la tabla intermedia PacienteServicio. Con esto logré que, al editar un paciente, aparezca una tabla en la parte inferior donde puedo registrarle nuevos servicios médicos, indicando la fecha y su estado ('Completado' o 'Pendiente') mediante columnas dinámicas editables sin tener que salir del formulario principal.

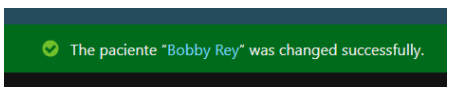


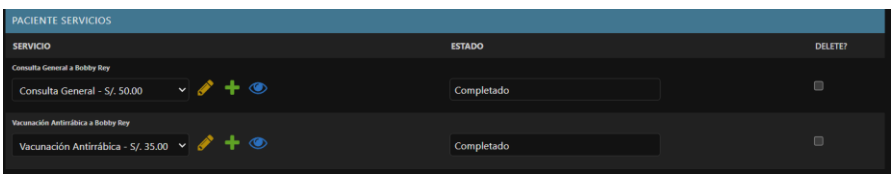
Ejercicio 13 — Verificar el flujo completo

Desde el panel del Admin, cree, edite y elimine al menos un registro relacionado (usando los Inlines). Documente con capturas el antes y el después, y compruebe que los cambios persisten en SQLite.

Ingresé al panel del administrador y realicé un flujo de prueba completo: agregué un nuevo paciente de mi investigación, le llené sus datos médicos en el inline apilado y le asigné un servicio desde la vista tabular. Luego, edité el estado del servicio y, finalmente, eliminé el registro para comprobar que las eliminaciones en cascada funcionen correctamente en mi base de datos SQLite. Todo el proceso fue exitoso y los cambios persistieron correctamente en el disco.







Ejercicio 14 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo qué modelos quedaron registrados en el Admin y cómo se personalizó cada uno (ModelAdmin, list_display, search_fields, list_filter, Inlines). Hagan commit y push al repositorio. Incluyan en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Generé el archivo requirements.txt actualizado y redacté los cambios en el README.md detallando qué modelos quedaron registrados en esta sesión y cómo personalicé la interfaz con ModelAdmin y los respectivos Inlines. Finalmente, subí todo el código a mi repositorio en la rama principal.

```
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_05> git push origin main
Enumerating objects: 32, done.
Counting objects: 100% (32/32), done.
Delta compression using up to 12 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (31/31), 10.31 KiB | 2.58 MiB/s, done.
Total 31 (delta 7), reused 6 (delta 1), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 1 local object.
To https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git
   45a9e0e..a852fe1  main -> main
```

URL del repositorio: https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de admin.py y el panel del Django Admin en el navegador).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

- **La implementación del Django Admin me demostró que no es necesario construir vistas y plantillas desde cero para el Backoffice de una aplicación; el framework reutiliza los modelos y el ORM que ya programé para generar un panel de gestión seguro e inmediato.**

- **Utilizar herramientas como StackedInline y TabularInline optimiza drásticamente el tiempo de administración de los datos, ya que permite al personal de la clínica gestionar las relaciones 1:1 y N:M desde una sola pantalla sin perder el contexto del paciente.**

- **El panel de administración es una herramienta exclusiva para el staff; seguiré necesitando programar mis propias Views y Templates para las interfaces que consumirán los clientes o usuarios finales de la clínica.**